registrii → „variabile" ale procesorului

Definirea variabilelor în RAM:

nume: .tip valoare

Tipuri de date:

- long - pt stocarea întregilor pe 4B (32 biți)
- word - pt stocarea întregilor pe 2B (16 biți)
- byte - pt stocarea ~~întregilor~~ unui caracter pe 1B (8 biți)
- ascii - pt stocarea șirurilor de caractere FĂRĂ terminator ("\n")
- asciz - pt stocarea șirurilor de caractere CU terminator ("\n")

} =>

=> fiecare caracter ocupă 1B (8 biți) - și spațiul se ~~pune~~ consideră caracter

- space N: alocă un spațiu de N B (8N biți)
- quad - alocă 8B (64 biți)

Scanat cu CamScanner

exercițiu cu tipuri de date: Cât ocupă fiecare variabilă

a) x_0: .quad 0x214

↳ 8B indiferent de valoare

b) x_1: .long 64

↳ 4B indiferent de valoarea dată

c) x_2: .space 64

↳ N = 64 => 64B

d) x_3: .space 8

↳ N = 8 => 8B

e) x_4: .byte 'a'

↳ 1B

f) x_5: .asciz "my␣Str␣" (1 2 3 4 5 6 7) + "\0" (8) => 8B - 1B pt fiecare

␣ -> spațiu

asciz este cu terminator

g) x_6: .ascii "my␣Str␣" (1 2 3 4 5 6 7) -> 7B -> 1B pt fiecare

h) x_7: .asciz "my␣Str␣\n" (1 2 3 4 5 6 7 8) + "\0" (9) -> 9B

! Dacă apare \n în șir se iau ambele caractere ca unul singur și nu are treabă cu terminatorul de șiruri dat de asciz

i) x_8: .ascii "my␣str␣\n" (1 2 3 4 5 6 7 8) -> 8B

Scanat cu CamScanner

Instrucțiunea MOV :

mov sursă, destinație

SURSA POATE FI :	DESTINAȚIA POATE FI :
ex. $4 (constantă)	ex. y (variabilă)
%eax (registru)	%ebx (registru)
x (variabilă)	

! variabilele din cod nu se trec cu $, doar const. iar %eax, %ebx sunt exemple. Se pot folosi orice registrii.

Uneori se poate întâlni mov cu sufixele :

- "b" (byte) - 1B, ex. movb
- "w" (word) - 2B, ex. movw
- "l" (long) - 4B, ex. movl
- "q" (quad) - 8B, ex. movq

Sufixele indică dimensiunea datelor asupra cărora acționează instrucțiunea. Deoarece în regiștri sau în memorie datele pot avea mărimi diferite, asamblorul are nevoie de această clarificare pentru a ști exact câți octeți trebuie să citească sau să scrie.

Dacă nu se pune sufixul, ~~nu se în~~ o să modifice sau o să citească toți ~~ceilalți~~ biții.

Scanat cu CamScanner

Apeluri de sistem

int $0x80 → syscall → cod întrerupere sistem

↳ întrerupțiom

EXIT → cod = 1

```
mov $1, %eax   → setează codul sistemului pt sys_exit
mov $0, %ebx   → codul de return (0 = fără erori)
int $0x80
```

WRITE → cod = 4

```
mov $4, %eax   → nr. apel sistem pt WRITE
mov $1, %ebx   → descriptor de fișier (1 = stout/ecran)
mov $str, %ecx → adresa str-ului pe care vrem să-l scriem
mov $(spațiul ~~lungimea~~ str-ului), %edx
int $0x80      → declanșarea efectivă a kernelului
```

Scanat cu CamScanner

Structura generală a unui program

- comentariu

```
.data
 # zona în care pot declara datele în memorie
 str : .asciz "Hello_, world!"
 x : .long 29

.text
 # începutul zonei de cod
.global _start
_start:
    mov $4, %eax
    mov $1, %ebx
    mov $str, %ecx
    mov $7, %edx
    int $0x80

et_exit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

} scriem șirul str din .data pe ecran

} practic return 0;

Scanat cu CamScanner

Operații aritmetice

`add op2, op1` ⟶ (înseamnă) `op1 += op2`

`sub op2, op1` ⟶ `op1 -= op2`

`mul op` ⟶ %EAX = %EAX · op

↳ mereu este %eax

`div op` ⟶ (EDX : EAX) / op = (EDX : EAX)

↑ rest ↑ cât

La mul, dacă rezultatul este prea mare să fie reținut în EAX, acesta este reținut și în EDX:

(EDX : EAX) ⟵ EAX · op

→ rezultatul operației este $2^{32} \cdot EDX + EAX$

→ dacă rezultatul încape în EAX, se suprascrie EDX cu 0

! EDX se modifică mereu în urma înmulțirilor/împărțirilor.

mul/div → pt nr. naturale

imul/idiv → pt nr. întregi

add $1, %eax (=> inc %eax → (inc înseamnă incrementare)

add $1, var (=> incl var

sub $1, %eax (=> dec %eax → (dec înseamnă decrementare)

sub $1, var (=> decl var

Scanat cu CamScanner

Program pentru împărțirea / înmulțirea a 2 variabile x și y

```
.data
  x: .long 30          -> x = 30
  y: .long 7           -> y = 7
produs: .space 4
cât: .space 4          } se alocă spațiu pt variabile
rest: .space 4
```

%eax	%ebx	%ecx	%edx
30	7		0
produs = 210			2 = rest
30			
cât = 4			

```
.text
.global _start
_start:
   mov x, %eax
   mov y, %ebx
   mul %ebx
   mov %eax, produs
   mov x, %eax
   div %ebx   # (%edx, %eax)
                rest  cât
   mov %edx, rest
   mov %eax, cât
.exit_et
   mov $1, %eax
   mov $0, %ebx      } return 0;
   int $0x80
```

Scanat cu CamScanner